

The Complete Flow — Every Step in Detail

From the moment you click **Send** in the chat UI to the moment the answer appears, here is exactly what happens: where control goes, what work is done, and how the backend produces the answer.

Table of Contents

- 1. [The two user actions](#)
- 2. [Flow A — File upload \(click-by-click\)](#)
- 3. [Flow B — Ask a question \(click-by-click\)](#)
- 4. [The HTTP requests and responses \(exact payloads\)](#)
- 5. [Every error path, clearly](#)
- 6. [Full end-to-end sequence diagram](#)

1. The two user actions

The app has exactly **two buttons** that trigger the backend:

Button	What the user does	Which function it calls
Upload	Selects a file, clicks Upload	<code>process-file</code>
Send	Types a question, clicks Send	<code>ask-question</code>

Everything below traces control for both.

2. Flow A — File upload (click-by-click)

Step-by-step control flow

1. USER: clicks "Upload"



2. FLUTTER APP (frontend):

- reads the file, extracts its text
- shows a "processing..." spinner
- builds HTTP POST request



HTTP POST /functions/v1/process-file
body: { "content": "...", "file_name": "notes.txt" }



3. SUPABASE PLATFORM:

- routes the request to the process-file Edge Function
- Deno runtime starts the function and calls Deno.serve handler



4. process-file (Edge Function) – stage 1: REQUEST VALIDATION

- is it an OPTIONS request? → yes, return "ok" (CORS preflight)
- parse the JSON body
- is "content" a non-empty string? → no → return 400 {error}
- is "file_name" a non-empty string? → no → return 400 {error}



5. process-file – stage 2: CHUNK THE TEXT (chunkText)

- collapse all whitespace to single spaces
- split on spaces into words
- group words into chunks of ~2000 characters each
- every chunk is a complete piece of text (no cut words)
- if no chunks → return 400 {error}



6. process-file – stage 3: EMBED EVERY CHUNK (embedText, in a loop)

- read GEMINI_API_KEY from environment
- for each chunk: HTTP POST to Gemini:
POST generativelanguage.googleapis.com/v1beta/
models/gemini-embedding-001:embedContent
body: { content: { parts: [{ text: "<chunk>" }] },
outputDimensionality: 768 }
- Gemini replies with 768 numbers → the chunk's meaning-vector
- if the reply has ≠ 768 numbers → abort with error



7. process-file – stage 4: SAVE TO DATABASE

- read SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY from environment
- create a Supabase client with the service role key
- for each chunk: build a row: {content, embedding, file_name}
- INSERT all rows into the "documents" table
- insert failed → return 500 {error}



8. process-file – stage 5: REPLY TO THE FRONTEND

- respond: 200 OK { "stored": <number of chunks> }



9. FLUTTER APP:

- receives {stored: n}
- shows "Uploaded successfully (n chunks stored)"
- stops the spinner

What the `documents` table looks like after upload

id (uuid)	content (text)	embedding	file_name	created_at (timestamp)
3f1a...9e2b	"Vicky is a friendly cat..."	[0.12,-0.34,...]	notes.txt	2026-08-05 10:02:11
7c9b...11d4	"She likes to sleep..."	[-0.01,0.88,...]	notes.txt	2026-08-05 10:02:11
a2e4...55f0	"Tax rules for 2026..."	[0.45,0.02,...]	report.pdf	2026-08-05 10:05:33

Every chunk = one row. The `embedding` column (768 numbers) is what makes these rows searchable by meaning.

3. Flow B — Ask a question (click-by-click)

Step-by-step control flow

1. USER: types "who is Vicky?" and clicks SEND



2. FLUTTER APP (frontend):

- captures the typed question
- appends a "user bubble" to the chat
- shows a "thinking..." / typing indicator
- builds HTTP POST request

HTTP POST /functions/v1/ask-question
body: { "question": "who is Vicky?" }



3. SUPABASE PLATFORM:

- routes the request to the ask-question Edge Function
- Deno runtime calls Deno.serve handler



4. ask-question – stage 1: REQUEST VALIDATION

- OPTIONS request? → return "ok" (CORS)
- parse JSON body
- is "question" a non-empty string? → no → return 400 {error}



5. ask-question – stage 2: EMBED THE QUESTION (embedText)

- read GEMINI_API_KEY from environment
- HTTP POST to Gemini:
models/gemini-embedding-001:embedContent
body: { content: { parts: [{ text: "who is Vicky?" }] },
outputDimensionality: 768 }
- Gemini replies with 768 numbers → the question's meaning-vector
- ≠ 768 numbers? → abort with error



6. ask-question – stage 3: SIMILARITY SEARCH (RPC match_documents)

- read SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY
- call: supabase.rpc("match_documents", {
query_embedding: "[0.05,-0.42,...]",
match_count: 3 })
- Supabase REST (PostgREST) runs the SQL function in Postgres:
SELECT id, content, file_name,
1 - (embedding <=> query_embedding) AS similarity
FROM documents
ORDER BY embedding <=> query_embedding
LIMIT 3;
- Postgres:
 - computes distance from question vector to EVERY row's vector
 - sorts smallest-distance first
 - keeps the top 3 rows
- returns 3 rows (or fewer/zero if the table is empty)



7. ask-question – stage 4: DECIDE

- 0 rows returned? → return 200 {answer: "No relevant context was found. Upload a document first."}
- otherwise continue



```
8. ask-question – stage 5: BUILD THE CONTEXT (buildPrompt)
• take the 3 chunk texts (row.content)
• join them with a blank line between each
• assemble the prompt:
  "You are an assistant that answers questions using ONLY the
  provided context.

  Context:
  <chunk 1 text>

  <chunk 2 text>

  <chunk 3 text>

  Question: who is Vicky?

  Answer using only the context above. If the context does not
  contain the answer, say that you don't know."
```



```
9. ask-question – stage 6: GENERATE THE ANSWER (generateAnswer)
• HTTP POST to Gemini:
  models/gemini-flash-latest:generateContent
  body: { contents: [{ parts: [{ text: "<the prompt>" }] }],
        generationConfig: { temperature: 0.4 } }
• Gemini reads ONLY the context in the prompt
• Gemini replies with the answer text
• empty reply? → return empty string
```



```
10. ask-question – stage 7: REPLY TO THE FRONTEND
• respond: 200 OK { "answer": "Vicky is a friendly cat who likes..." }
```



```
11. FLUTTER APP:
• receives {answer: "..."}
• stops the typing indicator
• renders the answer in a chat bubble
• scrolls to the bottom
```

4. The HTTP requests and responses (exact payloads)

Request 1 — File upload

```
POST https://<your-project>.supabase.co/functions/v1/process-file
Headers:
  Content-Type: application/json

Body:
{
  "content": "Vicky is a friendly cat. She likes to sleep all day and eat treats.",
  "file_name": "notes.txt"
}
```

```
Response 200:
{
  "stored": 2
}
```

Request 2 — Ask a question

```
POST https://<your-project>.supabase.co/functions/v1/ask-question
Headers:
  Content-Type: application/json

Body:
{
  "question": "who is Vicky?"
}
```

```
Response 200:
{
  "answer": "Vicky is a friendly cat who likes to sleep all day and eat treats."
}
```

What the Edge Function sends to Gemini (hidden from the user)

Embedding call (from both functions):

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:embedContent?key=GEMINI_API_KEY
Body:
{
  "model": "models/gemini-embedding-001",
  "content": { "parts": [{ "text": "who is Vicky?" }] },
  "outputDimensionality": 768
}

Response:
{ "embedding": { "values": [0.05, -0.42, 0.18, ... 768 numbers] } }
```

Generation call (from ask-question only):

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent?key=GEMINI_API_KEY
Body:
{
  "contents": [{ "parts": [{ "text": "<the grounded prompt>" }] }],
  "generationConfig": { "temperature": 0.4 }
}

Response:
{ "candidates": [{ "content": { "parts": [{ "text": "Vicky is a friendly cat..." }] } } ] }
```

5. Every error path, clearly

Where it fails	What happens	Response the user sees
<code>content</code> or <code>file_name</code> missing / empty	<code>process-file</code> returns 400	<code>{"error": "content and file_name are required"}</code>
Text produced no chunks	<code>process-file</code> returns 400	<code>{"error": "content produced no chunks"}</code>
<code>GEMINI_API_KEY</code> not set	both functions throw	500 <code>{"error": "GEMINI_API_KEY is not set"}</code>
Gemini embedding API error	both functions throw	500 <code>{"error": "Embedding API error <status>: <detail>"}</code>
Embedding has wrong size	both functions throw	500 <code>{"error": "Unexpected embedding dimension: <n>"}</code>
Database insert fails	<code>process-file</code> throws	500 <code>{"error": "<postgres error>"}</code>
<code>question</code> missing / empty	<code>ask-question</code> returns 400	<code>{"error": "question is required"}</code>
No similar chunks found	<code>ask-question</code> responds normally	<code>{"answer": "No relevant context was found. Upload a document first."}</code>
Gemini generation error	<code>ask-question</code> throws	500 <code>{"error": "Generation API error <status>: <detail>"}</code>

6. Full end-to-end sequence diagram

```
sequenceDiagram
    autonumber
    participant User
    participant Flutter as Flutter Web App
    participant Sup as Supabase Platform (router)
    participant PF as process-file (Edge Function)
    participant AQ as ask-question (Edge Function)
    participant GE as Gemini Embedding API
    participant GG as Gemini Generation API
    participant DB as Postgres + pgvector (documents table + match_documents)

    Note over User,DB: == PART 1: FILE UPLOAD ==
    User->>Flutter: clicks Upload (file selected)
    Flutter->>Sup: POST /functions/v1/process-file {content, file_name}
    Sup->>PF: route request → run Deno.serve handler
    PF->>PF: validate content & file_name (400 if bad)
    PF->>PF: chunkText() → split text into ~2000-char pieces
    loop each chunk
        PF->>GE: POST gemini-embedding-001:embedContent(chunk)
        GE-->>PF: 768-dimension vector
    end
    PF->>DB: INSERT rows {content, embedding, file_name}
    DB-->>PF: inserted (n rows)
    PF-->>Sup: 200 {stored: n}
    Sup-->>Flutter: 200 {stored: n}
    Flutter-->>User: "Uploaded successfully (n chunks)"

    Note over User,DB: == PART 2: ASK A QUESTION ==
    User->>Flutter: types "who is Vicky?" → clicks Send
    Flutter->>Sup: POST /functions/v1/ask-question {question}
    Sup->>AQ: route request → run Deno.serve handler
    AQ->>AQ: validate question (400 if empty)
    AQ->>GE: POST gemini-embedding-001:embedContent("who is Vicky?")
    GE-->>AQ: 768-dimension question vector
    AQ->>DB: RPC match_documents(query_embedding, match_count=3)
    Note over DB: SELECT ... ORDER BY embedding <=> query_embedding LIMIT 3
    DB-->>AQ: top 3 closest chunks {id, content, file_name, similarity}
    alt 0 chunks returned
        AQ->>Flutter: 200 {answer: "No relevant context found..."}
        Flutter-->>User: shows the message bubble
    else chunks found
        AQ->>AQ: buildPrompt(context, question) → grounded prompt
        AQ->>GG: POST gemini-flash-latest:generateContent(prompt)
        Note over GG: reads ONLY the context in the prompt
        GG-->>AQ: answer text
        AQ->>Sup: 200 {answer: "Vicky is a friendly cat..."}
        Sup-->>Flutter: 200 {answer: "..."}
        Flutter-->>User: answer shown in chat bubble
    end
```

The 3 key ideas to remember

1. **Every piece of text becomes numbers** — files *and* questions are turned into 768-number vectors by the *same* Gemini embedding model, so they live in the same “meaning space”.
2. **Search = find the closest numbers** — `match_documents` ranks all stored rows by cosine distance to the question’s vector and keeps the top 3.
3. **The answer is grounded** — only those 3 chunk texts go into the prompt, and Gemini is explicitly told to answer *only* from them (and to say “I don’t know” otherwise). That’s what makes the answer trustworthy.